

A Small Chess AI: Learning Chess from Scratch *

Technical Report & Training Guide

Chess AI Project

August 23, 2026

Abstract

This report documents a small, fully self-contained artificial-intelligence project whose goal is to *teach a small neural network to play chess* using the same recipe that produced AlphaZero: reinforcement learning through self-play guided by a Monte Carlo Tree Search (MCTS), accelerated by a short phase of *supervised pretraining* on human games. The complete system — neural network, search algorithm, training loop, evaluation, and a browser dashboard — runs on a single desktop machine (CPU + one consumer GPU). We explain what the system does, where its data comes from, how training is carried out, how the model “thinks,” and how long training takes.

Contents

1	What This Project Is	2
2	How Data Is Collected and Which Dataset Is Used	2
2.1	Supervised phase (human data)	2
2.2	Reinforcement phase (self-generated data)	3
3	Board Representation and Move Encoding	3
3.1	Input: 18 binary planes	3
3.2	Output: 4672 actions	3
4	Architecture: A Residual Policy–Value Network	3
5	How the AI Works: Monte Carlo Tree Search	4
5.1	Search enhancements used	4
6	How Training Is Done	4
6.1	Stage 1: Supervised pretraining (imitation)	4
6.2	Stage 2: Self-play reinforcement learning (improvement)	5
6.3	Data augmentation	5
7	How Long Does Training Take?	5
7.1	Measured throughput (current, pipelined run)	5
7.2	Budget for the current run	5
7.3	Why so little data can still make it play	5
8	Current Work: The v4 Improvement Round	6
8.1	Diagnosis that motivated this round	6
8.2	Changes implemented	6
8.3	Engineering: using CPU and GPU simultaneously	6

*An AlphaZero-style self-play project

9	Expected Impact	7
10	Playing with the Model	7
11	Reproducing the Project	7

1 What This Project Is

Goal. The project trains a compact neural network to play chess without ever being given chess knowledge (openings, tactics, or evaluation functions). The only inputs are the *rules* of chess (via the python-chess library) and, during pretraining, a collection of human chess games. The network learns purely from data — first by imitating humans, then by playing against itself.

Why a “small” model? A deliberately small network is used so that all training, search, and playing fits comfortably on one consumer machine. The default architecture has approximately **760,000 parameters** (compare: AlphaZero used hundreds of millions). This keeps every self-play game a few seconds long and makes an overnight training run feasible.

Deliverables.

- `model.py` — residual policy–value neural network.
- `mcts.py` — Monte Carlo Tree Search with PUCT exploration.
- `selfplay.py` — parallel self-play game generation.
- `pretrain_supervised.py` — supervised training on human games.
- `train.py` — main reinforcement-learning loop.
- `dashboard/` — a Flask web dashboard to play against the model and watch training metrics live.

2 How Data Is Collected and Which Dataset Is Used

2.1 Supervised phase (human data)

- **Source:** the Lichess open database (<https://database.lichess.org/>).
- **Files used:** five monthly dumps of *standard*, *rated* games (January–May 2013), stored as compressed PGN (`.pgn.zst`); about 90 MB compressed in total.
- **Quality filter:** only games where *both* players are rated at least 1750 Elo are kept, so the network imitates competent play rather than average play.
- **Examples extracted:** approximately **900 000** board positions, each with the move that a human actually played and the final game result.
- **Rehearsal anchor:** a random subset of 40 000 supervised positions is stored alongside the model and permanently mixed into every later self-play minibatch (Section 8).

Preprocessing. Each position is encoded as an $18 \times 8 \times 8$ tensor of binary planes (Section 3). Each human move is mapped to one of 4672 integer action labels, and the game outcome is converted to a target value in $\{-1, 0, +1\}$ from the perspective of the side to move.

Novelty/curriculum data. The same PGN files are mined for “rare opening positions” — 3000 positions taken a few moves into games. Self-play games are allowed to start from these positions 85% of the time, so the network explores the vast space of chess positions that pure self-play from the initial position would almost never visit. This is a distinctive feature of this project: the reinforcement agent is *seeded with positions discovered by humans*.

2.2 Reinforcement phase (self-generated data)

Once the network has a basic grasp of chess, it plays games *against itself*. No external data is used; the positions and outcomes are generated by the network’s own search. Each self-play game yields, for every move, the board position, the search’s move-probability distribution (policy target), and the final game result (value target). These experience tuples are stored in a replay buffer of up to 100 000 positions.

3 Board Representation and Move Encoding

3.1 Input: 18 binary planes

The board is encoded as 18 planes of 8×8 bits:

Planes	Meaning
0–5	White pieces: pawn, knight, bishop, rook, queen, king
6–11	Black pieces: same order
12–15	Castling rights (W-K, W-Q, B-K, B-Q)
16	En-passant target square
17	Side to move (all ones if White to move)

3.2 Output: 4672 actions

Each square is associated with 73 move types:

- 56 “queen-style” moves: $8 \text{ directions} \times 7 \text{ distances}$;
- 8 knight moves;
- 9 under-promotions (knight, bishop, rook $\times 3$ directions).

Hence the policy head outputs a probability distribution over $64 \times 73 = 4672$ actions. This is the same encoding used by Leela Chess Zero.

4 Architecture: A Residual Policy–Value Network

The model is a convolutional residual network (like a mini-Leela):

- Input convolution: $18 \rightarrow 64$ channels (3×3).
- Ten residual blocks (64 channels), each with two 3×3 convolutions and batch normalization.
- *Policy head*: a 1×1 convolution to 73 channels, reshaped to 4672 logits.
- *Value head*: a 1×1 convolution to 1 channel, a fully connected layer to 64 units, then a single scalar through tanh, estimating the expected outcome in $[-1, 1]$ from the side-to-move’s perspective.

Total trainable parameters: **760,717**. The two heads are trained jointly (multi-task): the value head learns *who is winning*, the policy head learns *which moves are good*.

5 How the AI Works: Monte Carlo Tree Search

The network alone is not a player; it is a *sensor* that evaluates positions and suggests moves. It becomes a player when wrapped in a Monte Carlo Tree Search (MCTS). MCTS repeatedly simulates future lines of play in four phases:

1. **Selection.** Starting at the root position, the search descends the tree choosing, at each node, the child with the best UCB score (PUCT):

$$U(s, a) = Q(s, a) + c_{\text{puct}} P(s, a) \frac{\sqrt{N(s)}}{1 + N(s, a)},$$

where Q is the child’s mean value, P is the network’s prior probability for the move, and N counts visits. This balances *exploitation* (high Q) and *exploration* (high prior, few visits).

2. **Expansion.** At a leaf, the network evaluates the position and produces (p, v) : a move distribution and a value estimate. Child nodes are created with priors p .
3. **Evaluation.** If the leaf is a terminal position (checkmate, stalemate, etc.), the true result replaces the network estimate.
4. **Backpropagation.** The value is propagated up the tree, with the sign flipped at every ply (zero-sum game), updating visit counts and running value sums.

After, say, 100–200 simulations, the move is chosen by the *visit count* distribution at the root — in effect, the search has “thought ahead” using the network as a guide.

5.1 Search enhancements used

- **Dirichlet noise** at the root during self-play, to force exploration and avoid repetitive games.
- **Temperature:** early in self-play games moves are sampled (temperature 1), later the best move is chosen deterministically.
- **Principal variation (PV):** the dashboard displays the model’s planned line of play.

6 How Training Is Done

The project uses a two-stage training pipeline.

6.1 Stage 1: Supervised pretraining (imitation)

The network is trained on the $\approx 800\,000$ human positions. For each example the losses are:

$$\mathcal{L}_{\text{pol}} = -\log p_{\theta}(m \mid s), \quad \mathcal{L}_{\text{val}} = (v_{\theta}(s) - z)^2, \quad \mathcal{L} = \mathcal{L}_{\text{pol}} + \mathcal{L}_{\text{val}}.$$

Optimizer: Adam, cosine learning-rate schedule $10^{-3} \rightarrow 10^{-4}$. This stage gives the network immediate chess sense — it opens with **e4/d4**, develops pieces, and avoids gross blunders — in a few minutes on a GPU.

6.2 Stage 2: Self-play reinforcement learning (improvement)

The AlphaZero loop repeats:

1. **Self-play:** the current network (with MCTS) plays games against itself, possibly starting from human-curriculum positions. Each move records the position and the MCTS visit distribution π .
2. **Training:** a minibatch is drawn from the replay buffer and the network is updated with the AlphaZero loss:

$$\mathcal{L} = \mathbb{E}_s \left[-\pi^\top \log p_\theta(s) + (v_\theta(s) - z)^2 \right] + \lambda \|\theta\|^2.$$

3. **Evaluation:** every few iterations the network is tested against a baseline (a greedy policy opponent) to monitor improvement.

6.3 Data augmentation

Chess is symmetric across the vertical axis. With probability 1/2 each training position is *mirrored* (files $a \leftrightarrow h$), doubling the effective dataset and the variety of positions the network sees.

7 How Long Does Training Take?

7.1 Measured throughput (current, pipelined run)

On the project’s machine (Intel i5-12400F, 6 cores/12 threads; NVIDIA RTX 3060), with self-play overlapped on GPU training:

Stage	Cost	Wall-clock
Download & parallel parse (5 PGN files)	5 CPU cores	≈ 2 min
Supervised pretraining (900k pos., 3 epochs, GPU)	GPU	≈ 3 min
One pipelined iteration (12 games 200 grad. steps)	CPU+GPU	$\approx 45\text{--}55$ s

7.2 Budget for the current run

The current run fine-tunes from the freshly pretrained model with a reduced learning rate (3×10^{-4}) for 300 polish iterations (iterations 201–500 of the loop):

$$300 \text{ iterations} \times \approx 47 \text{ s} \approx 3.9 \text{ hours.}$$

End to end — download, parsing, pretraining, and fine-tuning — the whole improvement cycle completes in roughly **4.5–5 hours**, comfortably inside a single day.

7.3 Why so little data can still make it play

The pretrained model already plays sensible openings because it imitated hundreds of thousands of human positions. Self-play then refines it on positions and plans that humans rarely reach. The result is a *beginner to club-player strength* engine: clearly better than random, coherent in openings and tactics, but far from engines like Stockfish (which use enormous networks, terabytes of self-play, and far more search). Strength can be scaled up simply by training longer, searching more simulations per move, and using a larger network.

8 Current Work: The v4 Improvement Round

8.1 Diagnosis that motivated this round

After the first full fine-tune (500 self-play iterations), head-to-head play against its own pretrained starting point ended in an even score — but inspection revealed a subtle regression: the fine-tuned model opened with weak moves (e.g. 1.f3), while the pretrained model chose mainline openings (1.e4/1.d4). This is *catastrophic forgetting*: noisy low-search self-play targets gradually overwrote the clean supervised knowledge.

8.2 Changes implemented

1. **Elo-filtered imitation data.** Only games with both players rated ≥ 1750 are used; dataset expanded from 2 to 5 monthly dumps (800k $\rightarrow$ 900k positions of noticeably better quality).
2. **Longer pretraining.** Three epochs instead of two. The policy cross-entropy loss dropped from ≈ 2.35 to ≈ 2.06 — a direct, measurable gain in move-prediction quality.
3. **Anchor rehearsal (anti-forgetting).** 30% of every training minibatch is now drawn permanently from a frozen set of 40 000 human positions. Self-play can still teach novelty, but it can no longer erase what the model already knows.
4. **Gentle polish phase.** Fine-tuning now runs at learning rate 3×10^{-4} (one third of before) for 300 iterations — enough to adapt, not enough to bulldoze.
5. **Human-curriculum openings.** Self-play games start (85% of the time) from 3 000 rare positions mined from the same human games, exposing the agent to plans far outside mainstream theory.

8.3 Engineering: using CPU and GPU simultaneously

- **Pipelined loop.** While the GPU performs iteration N 's gradient steps, the six CPU worker processes already play iteration $N+1$'s games. Neither processor idles; wall-clock per iteration fell from ≈ 70 s to ≈ 47 s ($\approx 33\%$ faster). Self-play uses weights one iteration stale — a standard, harmless relaxation.
- **Parallel parsing.** The five PGN files are decompressed and parsed in five simultaneous processes instead of one after another.
- **cuDNN autotuning** for the fixed-size convolution workload.

9 Expected Impact

Change	Mechanism	Expected effect
Elo-filtered data	imitate stronger moves only	fewer bad habits learned; openings and tactics match competent play
Anchor rehearsal	supervised loss never disappears	opening quality is preserved through self-play (directly fixes the 1.f3 regression)
Lower fine-tune LR	small weight updates	retains pretrained skill while still adapting
Curriculum openings	diverse starting positions	broader positional experience; less overfitting to one mainline
Mirror augmentation	every position teaches twice	effectively doubled supervised data at zero cost
Pipelining	no processor idles	$\approx 40\%$ shorter wall-clock for the same amount of learning

In short: the previous round proved the pipeline works end to end; this round makes the training *faster* (pipelining), the data *better* (Elo filter), and the result *stable* (rehearsal). The measurable indicators to watch are the pretraining policy loss (already improved from ≈ 2.35 to ≈ 2.06) and, on the dashboard, whether opening play remains principled (e4/d4) throughout self-play rather than degrading.

10 Playing with the Model

- **Browser dashboard** (dashboard/server.py): <http://127.0.0.1:5000> — play with the mouse, see the model’s principal variation and top moves, browse checkpoints, and run evaluations.
- **Teach-me loop**: after you play a game, the model can *fine-tune on your own moves*, becoming a personal opponent that adapts to your style.
- **Terminal play** (play.py): type UCI moves such as e2e4.

11 Reproducing the Project

```
# setup
python -m venv .venv
.venv\Scripts\activate          # Windows
pip install -r requirements.txt

# pretrain on human games (GPU optional, ~5 min)
python pretrain_supervised.py

# self-play fine-tune (resumes from the pretrained model)
python run_training.py 6 --device auto --iterations 700

# dashboard
python dashboard\server.py      # then open http://127.0.0.1:5000
```

References

- [1] Silver, D. et al. “Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm” (AlphaZero), *Science*, 2018.
- [2] Leela Chess Zero: <https://lczero.org>.
- [3] Lichess open database: <https://database.lichess.org>.
- [4] python-chess library: <https://python-chess.readthedocs.io>.